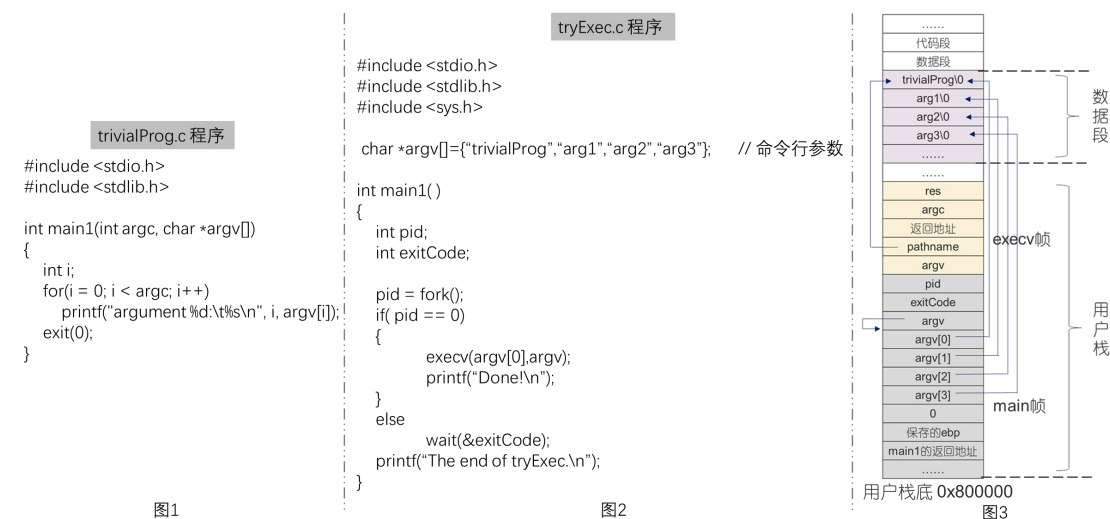


作业 6 exec 系统功能调用 和 shell 程序（指导）

同济大学计算机系操作系统 邓蓉 2025-11-21

一、解释这个程序的输出

命令行: #> tryExec



解题指导:

(1) 这个程序的输出是:

argument0: trivialProg

argument1: arg1

argument2: arg2

argument3: arg3

The end of tryExec.

(2) 因为子进程 `exec` 系统调用完成后, 进程图像中没有 `tryExec` 程序, 所以不会执行语句 `printf("Done!\n")`和 `printf("The end of tryExec.\n")`。

又因为子进程 `exec` 系统调用完成后, 进程图像中是 `trivialProg` 程序, 所以会输出 `tryExec` 提供的 4 个命令行参数。

最后, 父进程创建子进程后, `wait` 睡觉去了。所以父进程 `printf` 的字符行 `The end of tryExec.\n`, 显示在最后。

这个题, 要求完全掌握。

二、 修改 tryExec.c 程序，令 tryExec 进程执行系统调用时，用户栈如上图 3 所示。

解题指导：

```
#include <stdio.h>
#include <stdlib.h>
#include <sys.h>
int main1( )
{
    int pid;
    int exitCode;
    char *argv[]={“trivialProg”,“arg1”,“arg2”,“arg3”};

    pid = fork();
    if( pid == 0)
        execv(argv[0],argv);
    else
        wait(&exitCode);
    printf(“The end of tryExec.\n”);
}
```

其实，是 ppt 画图，一不小心画错了。这里写一遍，大家就全掌握啦。

请大家认真看 Exec()函数，内核从用户空间收集命令行参数这一段，非常精彩。

这个题，要求完全掌握。

三、 修改 tryExec.c，写一个简版 shell 程序 myShell.c。myShell 进程睡眠等待用户输入命令行，驱动命令行指定的任意应用程序，并且为它提供所需的命令行参数。要求程序在 Unix V6++ 系统里调出来。

解题指导：我们写一个 while 循环，要求支持外部命令 和 内部命令 logout。cd 命令选做。



最简版 shell 程序

命令行中只有一个程序。
fork 一个子进程，执行命令行中出现的程序。

```
While( )
{
    printf("$"); // 1、输出命令行提示符。
    scanf("%s", cmdLine); // 2、读取用户输入的命令行
    parse(cmdLine, argv); // 3、解析命令行，将命令行参数存入变长字符串数组 argv
    Swtch(argv[0]) { // 4、执行内部命令
        case "cd": // shell进程改变当前工作目录
        case "logout": // shell进程终止
        ..... 其它内部命令 .....
    }
    // 5、执行外部命令。即，为命令行中出现的每个应用程序创建一个子进程，初始化进程图像
    i = fork( ); // shell进程创建1个子进程
    if(i)
        wait( ); // shell进程睡眠等待子进程终止（子进程独用tty与用户交互）
    else
        exec(argv[0], argv); // 子进程执行应用程序
}
```

执行 exit 系统调用

四、把只读数据段移到共享正文段里。给出你的设计，包括、可能不限于改动后的内存描述符 MemoryDescriptor 对象，exec 系统调用 和 MapToPageTable 函数。写设计。调程序（选做）。